North American Private University | SFAX TUNISIA
TECHNOLOGY . BUSINESS . ARCHITECTURE

Université Nord-Américaine privée
Institut International de Technologie
Département Informatique

IIT INSTITUT INTERNATIONAL TECHNOLOGIE
Université Nord Américaine Privée

Travaux pratiques n°1	#Apprentissage des RNs profonds
Matière : Deep Learning	**Enseignants :** Taoufik Ben Abdallah
Discipline : 2ème année Génie Informatique	Tarak Ben Said
Année Universitaire : 2026-2027 / S1	

L'objectif de ce TP vise à construire un modèle de réseau de neurones pour classer les images de vêtements en utilisant la base de données **`fashion_mnist`**. Cette dernière représente un ensemble de données d'images d'articles de Zalando composé d'un ensemble d'apprentissage de **`60000`** exemples et d'un ensemble de test de **`10000`** exemples. Chaque exemple est une image en niveaux de gris **`28×28`**, associée à une **étiquette** de **`10`** **classes** **(`0 : T-shirt/top`, `1 : Trouser`, `2 : Pullover`, `3 : Dress`, `4 : Coat`, `5 : Sandal`, `6 : Shirt`, `7 : Sneaker`, `8 : Bag`, `9 : Ankle boot`)**

NB. Lancer Google Colab et téléchargez le fichier **`TP1_DL.ipynb`**. Ensuite, exécuter la première cellule du notebook qui vise à l'importation des bibliothèques, ainsi que la deuxième cellule qui tente à l'assemblage de votre compte Google Drive dans Colaboratory

Travail à faire :

1/ Importer la base de données **`fashion_mnist`** à partir du package **`datasets`** de **`tf.keras`**, puis charger les données d'apprentissage et de test dans **`4`** tableaux de type **ndarray** **`train_images`**, **`train_labels`**, **`test_images`**, et **`test_labels`** en utilisant ***`load_data`***

2/ Afficher la taille des tableaux **`train_images`**, **`train_labels`**, **`test_images`**, et **`test_labels`**

3/ Normaliser les valeurs des pixels de chaque image dans **`train_images`**, et **`test_images`** de sorte qu'on met les valeurs dans la même échelle de `0` à `1`

4/ Décomposer les données d'apprentissage en deux parties : **`90%`** sélectionnées aléatoirement sans répétions pour l'apprentissage (**`train_images`**, **`train_labels`**), et le reste pour la validation **`val_images`**, **`val_labels`**) (utiliser ***`train_test_split`*** de ***`sklearn.model_selection`***)
Afficher la taille des tableaux **`train_images`** et **`val_images`**

5/ Afficher les **`12`** premières images de l'ensemble d'apprentissage (Spécifier le nom de la classe sous chaque image)

6/ Construire un modèle de prédiction, nommé **`cls_fashion`**, en appliquant un réseau de neurones de **`5`** **couches** (une couche d'entrée, **`3`** couches cachées, et une couche de sortie). La **couche d'entrée** transforme le format des images d'un tableau à deux dimensions (**`28×28`** pixels) en un tableau à une dimension. Les deux premières **couches cachées** comportent **`128`** neurones tandis que la troisième couche cachée comporte **`64`** neurones.
Afficher un **résumé** de l'architecture de réseau de neurones de **`cls_fashion`**
NB.

- Activer les neurones des couches cachées par la fonction **`relu`**
- Configurer les couches de **`cls_fashion`** en utilisant la classe **`Sequential`** de **`tf.keras`**, la classe **`Flatten`** de **`tf.keras.layers`** pour configurer la couche d'entrée, et la classe **`Dense`** de **`tf.keras.layers`** pour configurer les couches cachées et de sortie)

7/ Compiler le modèle **`cls_fashion`** en appliquant l'**optimiseur** **`"adam"`** et entraîner le pour un **nombre d'époques** égal à **`10.`**

8/ Représenter la courbe d'évaluation de **`cls_fashion`** en fonction du nombre époques sur les données d'**apprentissage** et les données de **validation**

9/ Représenter la **matrice de confusion** associée aux données de test selon **`cls_fashion`**

10/ Calculer l'exactitude (**accuracy**) et la perte (**loss**) associées aux données de test selon **`cls_fashion`**

11/ Déduire le **rapport de classification** associée aux données de test selon **`cls_fashion`**

Bon Travail ♣